

Competitive Programming

Section 48: Divide And Conquer, Deque, And Knuth DP

Competitive Programming Course

2026-07-06

Table of contents

1	Optimization proof obligations	3
1.1	Divide-and-conquer DP optimization	3
1.2	Monotone deque DP	3
1.3	Knuth optimization	4
1.4	Red-level habit	4
1.5	Deque DP trace	4
1.6	Divide-and-conquer implementation shape	5
1.7	When to refuse an optimization	5
2	Exercises	6

1 Optimization proof obligations

DP optimizations are not magic templates. Divide-and-conquer optimization needs monotone optimal split points. Deque optimization needs a sliding transition window. Knuth optimization needs the interval split monotonicity property. In a contest, prove the condition or use a safer slower method.

1.1 Divide-and-conquer DP optimization

A common DP form is:

```
dp[g][i] = min over k < i of dp[g-1][k] + cost(k+1, i)
```

If the optimal split $\text{opt}[g][i]$ is nondecreasing as i increases, compute a whole row by divide and conquer:

```
solve(l, r, optL, optR):  
    mid = (l+r)/2  
    try k in [optL, optR]  
        recurse left with [optL, bestK]  
        recurse right with [bestK, optR]
```

The optimization is only correct if the monotonicity property holds. The local exercise uses a quadratic group cost where this property is valid.

1.2 Monotone deque DP

For:

```
dp[i] = a[i] + min(dp[j]) for i-w <= j < i
```

Maintain a deque of candidate indices with increasing dp values.

At position i :

1. Remove indices $< i-w$ from the front.
2. The front is the minimum.
3. Before inserting i , remove indices from the back while their dp is $\geq \text{dp}[i]$.

Every index enters and leaves once, so the total time is linear.

1.3 Knuth optimization

Interval DP often looks like:

```
dp[l][r] = min over k in [l,r) of dp[l][k] + dp[k+1][r] + cost(l,r)
```

Knuth optimization applies when optimal split points satisfy:

```
opt[l][r-1] <= opt[l][r] <= opt[l+1][r]
```

Then each state only searches a small range of candidate splits.

The classic adjacent merging problem with prefix-sum interval costs satisfies the needed quadrangle inequality. In practice, never apply Knuth just because the DP is interval-shaped; the monotonicity condition is the whole reason it works.

1.4 Red-level habit

For every optimization, write this before coding:

```
State:  
Transition:  
Naive complexity:  
Property that reduces candidates:  
Why the property is true:
```

1.5 Deque DP trace

Let:

```
dp[i] = a[i] + min(dp[j]) for i-3 <= j < i
```

The deque stores indices with increasing `dp`.

Before computing `i`, remove expired indices `< i-3`. The front is then the best valid `j`. After computing `dp[i]`, remove worse candidates from the back: if `dp[old] >= dp[i]`, then `old` will never beat `i` for any future window where both are present.

This dominance argument is the proof. Without it, a deque implementation is just a recipe.

1.6 Divide-and-conquer implementation shape

The recursive function computes one DP row:

```
compute(l, r, optL, optR)
```

It fills $mid = (l+r)/2$ by trying candidate splits from `optL` to `optR`. If the best split is `best`, monotonicity gives:

```
left half candidates <= best  
right half candidates >= best
```

If you cannot prove that the optimal split is monotone, this recursion may discard the real answer. Use the slower DP instead.

For Knuth optimization, compute by increasing interval length and search only:

```
opt[l][r-1] ... opt[l+1][r]
```

Clamp that range to valid split indices.

1.7 When to refuse an optimization

If constraints allow $O(n^2)$ and you cannot prove the monotonicity condition, submit the clear $O(n^2)$ DP. A wrong $O(n \log n)$ optimization is worse than a slower accepted solution.

For learning, keep both versions:

```
slow reference: easy to trust  
optimized version: compare by random testing
```

This is especially useful for divide-and-conquer DP and Knuth optimization.

2 Exercises

- A. Partition Quadratic Cost: Partition the array into k contiguous groups minimizing the sum of squared group sums.
- B. Window Minimum DP: $dp[i] = a[i] + \min(dp[j])$ over the previous w positions. Print $dp[n]$.
- C. Optimal Merge Knuth: Merge adjacent piles with cost equal to merged sum; print minimum total cost.